

Task 12. Simulate Gaming concepts using Pygame 1st/2nd

Aim: To simulate Gaming concepts using Pygame.

Snake Game.

Plan: 1. Write a Python program to create a snake game using pygame package.

Conditions.

1. set window size
2. create a snake.
3. make the snake to move in the directions when left, right, down and up key is pressed
4. when the snake hits the fruit, increase the score by 10.
5. If the snake hits the window, game over.

Algorithm:

1. Import Pygame package and initialize it
2. Define the window size and title
3. create a snake class which initializes the snake position, color, & movement.
4. create a fruit class which initializes the fruit position & color
5. create a function to check if the snake collides with the fruit and increase the score
6. create a function to check if the snake collides with the window and end the game.
7. create a function to update the snake position and
8. create a ~~function~~ to update the game display on the user input and draw the snake and fruit.
9. create a game loop to continuously update the game display, snake position, and clock for collisions.
10. End the game if the user quits or the snake collides with the window

Sample output.

(value 1: "value 1")

(value 2: "value 2")

snake game

score: 10

Submit

Run the main loop
 root.mainloop()

SL. NO.	NAME
1	...
2	...
3	...
4	...
5	...
6	...
7	...
8	...
9	...
10	...

Result: The program using Tkinter module for all design was executed successfully.

Program:

importing libraries

import pygame

import time

import random

Snake-speed = 15

window size

window-x = 720

window-y = 480

defining colors

black = pygame.Color(0, 0, 0)

white = pygame.Color(255, 255, 255)

red = pygame.Color(255, 0, 0)

green = pygame.Color(0, 255, 0)

blue = pygame.Color(0, 0, 255)

creating pygame

pygame.init()

creating game window

pygame.display.set_caption('Snake Game')
game-window = pygame.display.set_mode((window-x, window-y))

fps (frames per second) controller

fps = pygame.time.Clock()

defining snake default position

snake-position = (100, 50)

defining first 4 blocks of snake body

snake-body = [(100, 50),

(120, 50),

(140, 50),

(160, 50)

]

fruit position

fruit-position = [random.randrange(1, (window-x//10)) * 10,
random.randrange(1, (window-y//10)) * 10]

fruit-spawn = True

setting default snake direction towards

right

direction = 'RIGHT'

change-to = direction

initial score

score = 0

displaying score function

def show-score(choice, color, font, size):

creating the display ~~for~~ object score-font

score-font = pygame.font.SysFont(font, size)

create the display surface object

score-surface

score-surface = score-font.render('score: ' + str(score), True,
color)

create a rectangular object for the text

surface object

score-rect = score-surface.get_rect()

displaying text

game-window.blit(score-surface, score-rect)

~~# game over function~~

~~def game-over():~~

~~# creating font object my-font~~

~~my-font = pygame.font.SysFont('times new roman', 50)~~


```
#creating a text surface on which text  
# will be drawn  
game_over_surface = my_font.render(  
    'Your score is: ' + str(score), True, red)
```

```
# create a rectangular object for the text  
# surface object  
game_over_rect = game_over_surface.get_rect()
```

```
# setting position of the text  
game_over_rect.midtop = (window_x/2, window_y/4)
```

```
# blit will draw the text on screen.  
game_window.blit(game_over_surface, game_over_rect)  
pygame.display.flip()
```

```
# after 2 seconds we will quit the program  
time.sleep(2)
```

```
# deactivating pygame library  
pygame.quit()
```

```
# quit the program  
quit()
```

```
# main function  
while True:
```

```
# handling key events
```

```
for event in pygame.event.get():
```

```
    if event.type == pygame.KEYDOWN:
```

```
        if event.key == pygame.K_UP:  
            change_to = 'up'
```

```
        if event.key == pygame.K_DOWN:
```

```
            change_to = 'down'
```


if event.key == pygame.K_LEFT:

change_to = 'LEFT'

if event.key == pygame.K_RIGHT:

change_to = 'RIGHT'

if two keys pressed simultaneously

we don't want snake to move into two

directions simultaneously

if change_to == 'UP' and direction != 'DOWN':
direction = 'UP'

if change_to == 'DOWN' and direction != 'UP':
direction = 'DOWN'

if change_to == 'LEFT' and direction != 'RIGHT':
direction = 'LEFT'

if change_to == 'RIGHT' and direction != 'LEFT':
direction = 'RIGHT'

moving the snake

if direction == 'UP':

snake_position[1] -= 10

if direction == 'DOWN':

snake_position[1] += 10

if direction == 'LEFT':

snake_position[0] -= 10

if direction == 'RIGHT':

snake_position[0] += 10

snake body growing mechanism

if snakes are snakes collide then

will be incremented by 10

snake-body, insert (0, list(snake-position))

if snake-position[0] == fruit-position[0] and

snake-position[1] == fruit-position[1]:

score += 10

fruit-spawn = false

else:

snake-body.pop()

if not fruit-spawn:

fruit-position = [random.randrange(0, (windows-1/10)) * 10,
random.randrange(0, (windows-1/10)) * 10]

fruit-spawn = True

game-windows += 1 (Halt)

for pos in snake-body:

pygame.draw.rect (game-windows, green,

pygame.rect (pos[0], pos[1], 10, 10))

pygame.draw.rect (game-windows, white, pygame.rect(
fruit-position[0], fruit-position[1], 10, 10))

Game over condition

if snake-position[0] < 0 or snake-position[0] > windows-10:
game-over()

if snake-position[1] < 0 or snake-position[1] > windows-10:
game-over()

displaying score continuously

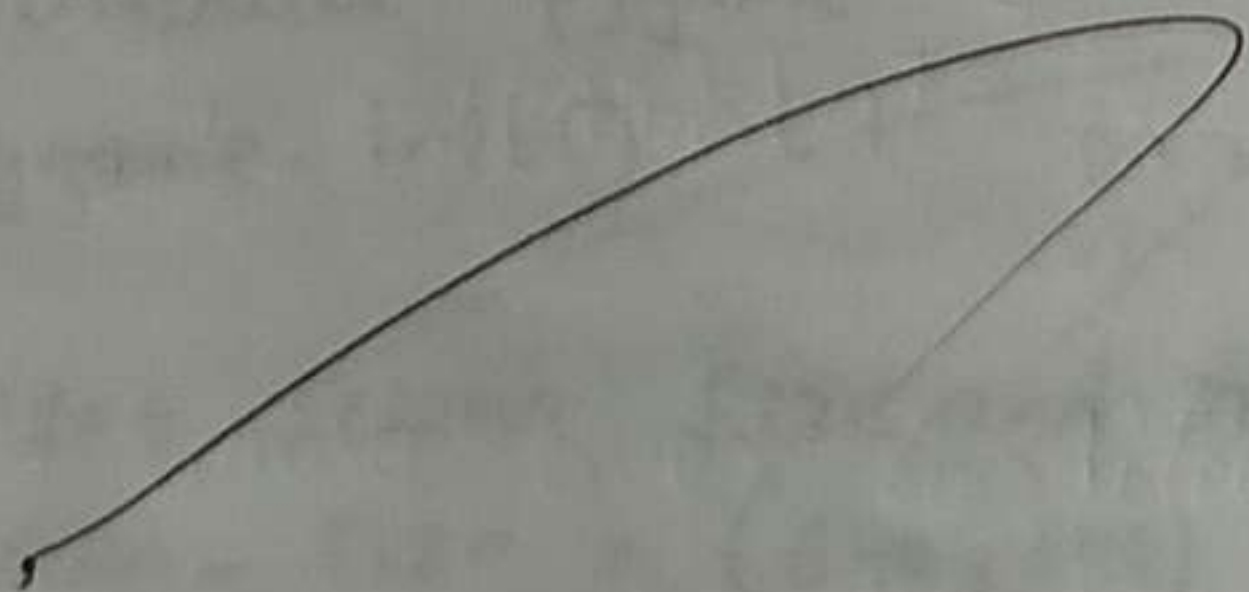
show_score (1, white, 'times new roman', 20)

refresh game screen

pygame.display.update()

frame per second / refresh rate

FPS.tick (snake-speed)



2. write a python program to Develop a chess board using pygame.

Algorithms:-

1. Import Pygame and initialize it.
2. Set screen size and title.
3. Define colors for the board and pieces.

Define a function to draw the board by looping over rows and columns and drawing squares of different colors.

4. Define a function to draw the pieces on the board by loading images for each piece and placing them on the corresponding square.
5. Define the initial state of the board as a list of lists containing the pieces.
6. Draw the board and pieces on the screen.
7. Start the game loop.

Program:-

```
import pygame
```

```
# initialize pygame  
pygame.init()
```

```
# set screen size and title  
screen_size = (640, 640)
```

```
screen = pygame.display.set_mode(screen_size)  
pygame.display.set_caption('chess Board')
```


Define Colors

black = (0, 0, 0)

white = (255, 255, 255)

brown = (153, 76, 0)

Figure 31qms2

Define function to draw the board.

def draw_board():

for row in range(8):

for col in range(8):

square_color = white if (row + col) % 2 == 0 else brown

square_rect = pygame.Rect(col * 80, row * 80, 80, 80)

pygame.draw.rect(screen, square_color, square_rect)

Define function to draw the pieces

def draw_pieces(board):

piece_images = {

'r': pygame.image.load('images/rook.png'),

'n': pygame.image.load('images/knight.png'),

'b': pygame.image.load('images/bishop.png'),

'q': pygame.image.load('images/queen.png'),

'k': pygame.image.load('images/king.png'),

'p': pygame.image.load('images/pawn.png'),

}

for row in range(8):

for col in range(8):

piece = board[row][col]

if piece != '':


```
piece_image = piece_images[piece]
```

```
piece_rect = pygame.Rect(col*80, row*80, 80, 80)
```

```
screen.blit(piece_image, piece_rect)
```

```
# Define initial state of the board.
```

```
board = [
```

```
['x', 'n', 'b', 'q', 'k', 'b', 'n', 'x'],
```

```
['p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'],
```

```
['', '', '', '', '', '', '', ''],
```

```
['', '', '', '', '', '', '', ''],
```

```
['', '', '', '', '', '', '', ''],
```

```
['', '', '', '', '', '', '', ''],
```

```
['p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'],
```

```
['r', 'n', 'b', 'q', 'k', 'b', 'n', 'r']
```

```
]
```

```
# Draw board and pieces
```

```
draw_board()
```

```
draw_pieces(board)
```

```
# start game loop
```

```
while True:
```

```
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
```

```
            pygame.quit(),
```

```
            quit()
```

```
        pygame.display.update()
```


Define function to draw the board
 def draw_board():
 board = [[0,0,0], [0,0,0], [0,0,0]]
 print(board)

Sample output:-

Define function to draw the board
 def draw_board():
 board = [[0,0,0], [0,0,0], [0,0,0]]
 print(board)

VEL TECH	
EE No.	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
MARKS (5)	
RECORD (5)	
TOTAL (30)	
DATE	

Result:

Thus the program for pygame is executed and verified successfully.